

Práctico AYED1 - 31-08 - Sala 2

Práctico 1 - Ejercicio 14c

Aplicar, si es posible, cambio de variable:

$$\langle \prod i : 1 \leq i < \#(x \blacktriangleright xs) : (x \blacktriangleright xs) ! i \rangle$$

con $f.j = j + 1$. Si se pudiera aplicar, quedaría:

$$\langle \prod j : 1 \leq j + 1 < \#(x \blacktriangleright xs) : (x \blacktriangleright xs) ! (j + 1) \rangle$$

¿Qué dice la regla de cambio de variable? Dice que podemos reemplazar i por $f.j$ siempre que f sea una función biyectiva entre los conjuntos: de salida $S = \{ j \mid R.(f.j) \}$ y de llegada $L = \{ i \mid R.i \}$.

Observación:

- El conjunto L es el conjunto de valores que satisfacen el rango original (cuando hacemos la "lectura operacional").
- El conjunto S es el conjunto de valores que satisfacen el rango que queda al aplicar la regla.

En este caso:

- $R.i \equiv 1 \leq i < \#(x \blacktriangleright xs) = \#xs + 1$. Luego $i \in L = \{ 1, 2, \dots, \#xs-1, \#xs \}$. (ejemplo, si $\#xs = 4$, $i \in \{1, 2, 3, 4\}$).
- $R.(f.j) \equiv 1 \leq j + 1 < \#(x \blacktriangleright xs) = \#xs + 1$. Luego $j \in S = \{ 0, 1, \dots, \#xs-2, \#xs-1 \}$. (ejemplo, si $\#xs = 4$, $j \in \{0, 1, 2, 3\}$).
- La función $f.j = j + 1$ es una biyección entre S y L :
 $S \rightarrow L$
 $0 \rightarrow 1$

1 -> 2

2 -> 3

3 -> 4

- $f.j = j + 1$ **siempre es biyectiva** por lo tanto siempre se puede aplicar cambio de variable con esta f.

Luego, **sí se puede aplicar el cambio de variable.** Lo escribiremos así:

$\langle \prod i : 1 \leq i < \#(x \blacktriangleright xs) : (x \blacktriangleright xs) ! i \rangle$

= { cambio de variable con $f.j = j + 1$ (o podemos decir $j \rightarrow j + 1$) }

$\langle \prod j : 1 \leq j + 1 < \#(x \blacktriangleright xs) : (x \blacktriangleright xs) ! (j + 1) \rangle$

= { aplicamos un par de pasos más sólo para simplificar: def. de # }

$\langle \prod j : 1 \leq j + 1 < \#xs + 1 : (x \blacktriangleright xs) ! (j + 1) \rangle$

= { algebra }

$\langle \prod j : 0 \leq j < \#xs : (x \blacktriangleright xs) ! (j + 1) \rangle$

= { def de ! }

$\langle \prod j : 0 \leq j < \#xs : xs ! j \rangle$

Práctico 1 - Ejercicio 14d

Aplicar, si es posible, cambio de variable:

$\langle \text{Max } as : as \neq [] : \#as \rangle$ con $f.(b, bs) = b \blacktriangleright bs$

Supongamos, por simplicidad, $as \in [\text{Int}]$.

Si se pudiera aplicar el cambio de variable, quedaría:

$\langle \text{Max } b, bs : b \blacktriangleright bs \neq [] : \#(b \blacktriangleright bs) \rangle$

Observación: vale hacer cambio de variable cambiando una sola variable (as) por 2 variables (b, bs). Siempre que se mantenga la biyectividad entre los conjuntos. Ahora, el conjunto de salida S será un conjunto de pares.

- Conjunto de llegada: $L = \{ [-11], [1], \dots, [0, 0], [11, 22], \dots [34, 4, 12], [-343, -12, 1], \dots \}$ (todas las listas posibles menos $[]$)
- Conjunto de salida. Miramos el predicado $b \blacktriangleright bs \neq [] \equiv \text{True}$. $(b, bs) \in S = \{ (2, []), (345, [0, -10]), \dots \}$ (todos los pares posibles $\text{Int } x \times [\text{Int}]$)
- Es biyectiva f entre estos dos conjuntos?
 $S \rightarrow L$
 $(-11, []) \rightarrow [-11]$
 $(1, []) \rightarrow [1],$
 $\dots$
 $(0, [0]) \rightarrow [0, 0]$
 $(11, [22]) \rightarrow [11, 22]$
 $\dots$
 $(34, [4, 12]) \rightarrow [34, 4, 12]$
 $(-343, [-12, 1]) \rightarrow [-343, -12, 1]$
- Vemos que f sí es biyectiva entre estos conjuntos y por lo tanto **se puede** aplicar el cambio de variable.
- Pregunta: ¿Qué hubiera pasado si $[] \in L$? En este caso, no hay ningún elemento de S que me lleve a la lista vacía a partir de la función f. No sería suryectiva, y luego no sería biyectiva.
- **Observación:** $f.(b, bs) = b \blacktriangleright bs$ es biyectiva siempre que en el rango original (o sea conjunto de llegada L) no esté la lista vacía. En estos casos siempre se puede aplicar el cambio de variable.

Aplicamos el cambio de variable y un par de pasos más:

$\langle \text{Max } as : as \neq [] : \#as \rangle$
 $= \{ \text{cambio de variable con } f.(b, bs) = b \blacktriangleright bs \text{ (o también se puede decir } as \rightarrow b \blacktriangleright bs) \}$
 $\langle \text{Max } b, bs : b \blacktriangleright bs \neq [] : \#(b \blacktriangleright bs) \rangle$
 $= \{ \text{prop. listas} \}$
 $\langle \text{Max } b, bs : \text{True} : \#(b \blacktriangleright bs) \rangle$
 $= \{ \text{def. } \# \}$
 $\langle \text{Max } b, bs : \text{True} : \#bs + 1 \rangle$

Práctico 1 - Ejercicio 14z

¿Se puede aplicar cambio de variable en este caso?

$\langle \text{Max } as : \#as \neq 5 : \#as \rangle$ con $f.(b, bs) = b \blacktriangleright bs$

No se puede porque $[] \in L$ y f no es biyectiva en ese punto.

Práctico 1 - Ejercicio 15b

Aplicar alguna de las reglas de conteo:

$\langle N i : i - n = 1 : \text{par}.i \rangle$ a donde $\text{par} : \text{Int} \rightarrow \text{Bool}$ me dice si un número es par.

Veamos:

$\langle N i : i - n = 1 : \text{par}.i \rangle$
 $= \{ \text{aritmetica} \}$
 $\langle N i : i = 1 + n : \text{par}.i \rangle$

= { rango unitario }

~~par.(1+n)~~ -> MAL MUY MAL: ni siquiera tipa, teníamos un Int y ahora es un Bool.

(par.(1+n) → 1

[] ¬ par.(1+n) → 0

)

Práctico 2 - Ejercicio 2a

Dar el tipo y derivar a partir de la especificación:

a) $\text{sum_cuad.xs} = \langle \sum i : 0 \leq i < \#xs : xs[i] * xs[i] \rangle$

Ejemplo: con $xs = [-2, 7, 0, 3, 1]$. $\text{sum_cuad.xs} = (-2)*(-2) + 7*7 + 0*0 + 3*3 + 1*1 = 63$.

Tipo: $\text{sum_cuad} : [\text{Int}] \rightarrow \text{Int}$.

Derivación: La haremos por inducción en xs . (porqué? porque tenemos q calcular un cuantificador)

- Caso base: $xs = []$.

$\text{sum_cuad}[]$

= { especificación }

$\langle \sum i : 0 \leq i < \#[] : [][i] * [][i] \rangle$

= { def. de # }

$\langle \sum i : 0 \leq i < 0 : [][i] * [][i] \rangle$

= { lógica, aritmetica, algebra, qué se yo }

$$\langle \sum i : \text{False} : []!i * []!i \rangle$$

$$= \{ \text{rango vacío} \}$$

$$0$$

(llegamos a algo que está en el lenguaje de programación (la ctte. 0), por lo tanto terminamos este caso).

- Paso inductivo: $xs = y \triangleright ys$ (NUNCA escribir $xs = x \triangleright xs$ (esto es False))

Queremos obtener una definición para $\text{sum_cuad.}(y \triangleright ys)$, asumiendo la hipótesis inductiva (HI):

$$\text{sum_cuad.}ys = \langle \sum i : 0 \leq i < \#ys : ys!i * ys!i \rangle.$$

Veamos:

$$\text{sum_cuad.}(y \triangleright ys)$$

$$= \{ \text{especificación} \}$$

$$\langle \sum i : 0 \leq i < \#(y \triangleright ys) : (y \triangleright ys)!i * (y \triangleright ys)!i \rangle$$

$$= \{ \text{def. de } \# \}$$

$$\langle \sum i : 0 \leq i < \#ys + 1 : (y \triangleright ys)!i * (y \triangleright ys)!i \rangle$$

$$= \{ \text{lógica: } 0 \leq i < \#ys + 1 \equiv i = 0 \vee 1 \leq i < \#ys + 1 \}$$

(ojo, vale porque $\#ys + 1 \geq 1$, si no no valdría, ver ejercicio P1-11) }

$$\langle \sum i : i = 0 \vee 1 \leq i < \#ys + 1 : (y \triangleright ys)!i * (y \triangleright ys)!i \rangle$$

$$= \{ \text{partición rango (ya que los rangos son disjuntos)} \}$$

$$\langle \sum i : i = 0 : (y \triangleright ys)!i * (y \triangleright ys)!i \rangle + \langle \sum i : 1 \leq i < \#ys + 1 : (y \triangleright ys)!i * (y \triangleright ys)!i \rangle$$

$$= \{ \text{rango unitario} \}$$

$$(y \triangleright ys)!0 * (y \triangleright ys)!0 + \langle \sum i : 1 \leq i < \#ys + 1 : (y \triangleright ys)!i * (y \triangleright ys)!i \rangle$$

$$= \{ \text{def. !} \}$$

$$y * y + \langle \sum i : 1 \leq i < \#ys + 1 : (y \triangleright ys)!i * (y \triangleright ys)!i \rangle$$

$$= \{ \text{cambio de variable con } f.j = j + 1 \text{ (pero reusamos la letra } i \text{)} \}$$

$$y * y + \langle \sum i : 1 \leq i + 1 < \#ys + 1 : (y \triangleright ys)!(i + 1) * (y \triangleright ys)!(i + 1) \rangle$$

$$= \{ \text{def. !} \}$$

$$y * y + \langle \sum i : 1 \leq i + 1 < \#ys + 1 : ys!i * ys!i \rangle$$

$= \{ \text{aritmética, cursillo, o q se yo: restamos 1 en los miembros de la desigualdad} \}$
 $y * y + \langle \sum i : 0 \leq i < \#ys : ysl i * ysl i \rangle$
 $= \{ \text{H. I.} \}$
 $y * y + \text{sum_cuad.ys}$
 (acá terminamos, ya que llegamos a algo que está en el lenguaje de programación)

Resultado de la derivación:

$\text{sum_cuad} : [\text{Int}] \rightarrow \text{Int}$
 $\text{sum_cuad}[] \doteq 0$
 $\text{sum_cuad}(x \blacktriangleright xs) \doteq x * x + \text{sum_cuad}.xs$

Volvemos a esto un momento: NUNCA escribir $xs = x \blacktriangleright xs$ (esto es False). Desafío: dar x, xs tales que vale $xs = x \blacktriangleright xs$.
 No existe. Importante: Entender que la igualdad "=" es un predicado, devuelve un booleano. No es "reemplazo esto por aquello".

¿Porqué hacemos inducción? Veamos un ejemplo en el que no hace falta inducción:

Especificación:

$\text{agrega354} : [\text{Int}] \rightarrow [\text{Int}]$
 $\text{agrega354}.xs = 354 \blacktriangleright xs$

Derivar: O sea, encontrar una definición ("programa") que calcula agrega354 . Podemos hacer directo, sin inducción:

$\text{agrega354}.xs$
 $= \{ \text{especificación} \}$
 $354 \blacktriangleright xs$

(fin de la derivación ya que tengo algo que está en el lenguaje de programación).

Resultado:

$\text{agrega354}.xs \doteq 354 \blacktriangleright xs$

Práctico 2 - Ejercicio 2b

No vamos a hacer la derivación, intentemos dar la definición intuitivamente:

$$\text{iga.e.xs} = \langle \forall i : 0 \leq i < \#xs : xs[i] = e \rangle$$

¿Qué calcula iga? Me dice si todos los elementos de xs son iguales a e.

$$\text{iga.34}.[34, 34, 12, 34] = \text{False}$$

$$\text{iga.4}.[4, 4, 4] = \text{True}$$

Definición:

$$\text{iga} : A \rightarrow [A] \rightarrow \text{Bool}$$

$$\text{iga.e}.[] \doteq \text{True}$$

$$\text{iga.e}.(x \blacktriangleright xs) \doteq (x = e) \wedge \text{iga.e.xs}$$

Ejercicio: derivar!